

FORMAL LANGUAGES AND COMPILERS

prof.s Giovanni Agosta, Luca Breveglieri and Angelo Morzenti

Exam of MONDAY 6 FEBRUARY 2023 – Theory Part

WITH SOLUTIONS – FOR TEACHING PURPOSES THE SOLUTIONS ARE WIDELY COMMENTED

LAST NAME (SURNAME) + FIRST NAME:

(capital letters please)

MATRICOLA:

(or PERSON CODE)

SIGNATURE:

TEACHER: ☐ Prof. G. AGOSTA – ☐ Prof. L. BREVEGLIERI – ☐ Prof. A. MORZENTI

INSTRUCTIONS - READ CAREFULLY:

- The exam is in written form and consists of two parts:
 1. Theory (80%): syntax and semantics of languages, divided in four sections:
 - regular expressions and finite automata
 - free grammars and pushdown automata
 - syntax analysis and parsing methodologies
 - language translation and semantic analysis
 2. Lab (20%): compiler design by Flex and Bison
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- For the theory part to be valid, the candidate must achieve a grade of at least 10/20 on each of the four sections.
- Correctly answering all the questions that are not marked optional, allows the candidate to achieve a high grade (although not the full mark).
- For the lab part to be valid, the candidate must achieve a grade of at least 15/30.
- The final grade is the weighted average of the theory part (80 %) and of the lab part (20 %), and must be $\geq 18/30$.
- The exam is open book: textbooks and personal notes are permitted.
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: lab part 1 h – theory part 2 h.

1 Regular Expressions and Finite Automata 20%

1. Consider the regular expression R below, over the two-letter alphabet $\Sigma = \{a, b\}$

$$R = ((a^+ b^*)^* \mid b a)^+$$

Answer the following questions:

- (a) Write a derivation of the regular expression R for the string $\varepsilon \in L(R)$.
 - (b) Write all the strings of length exactly 3 that are generated by the regular expression R .
 - (c) By using the Berry-Sethi method, find a deterministic automaton A equivalent to the regular expression R .
 - (d) Determine if the automaton A is minimal and provide an explanation, and if necessary find the corresponding minimal automaton.
 - (e) (optional) Starting from the automaton A , possibly in the minimized version, write a right-unilinear grammar G equivalent to the regular expression R .
-

Solution

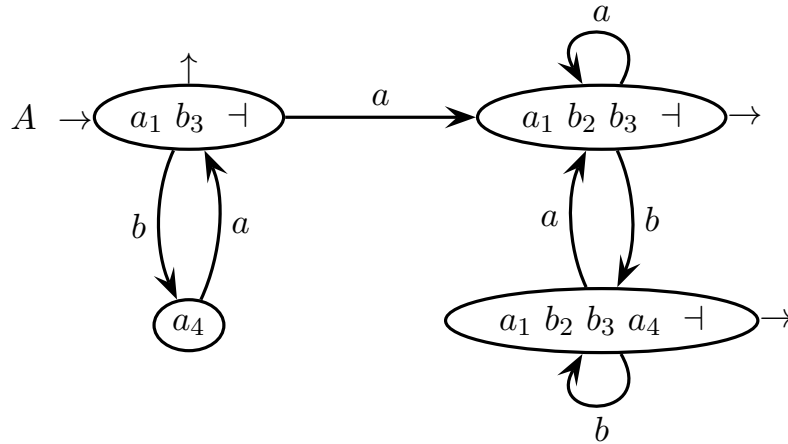
- (a) The regular expression R generates four strings of length equal to 3. Here they are (listed in alphabetical order):

$a a a \quad a a b \quad a b a \quad a b b$

- (b) Here is the Berry-Sethi (deterministic) automaton A equivalent to the regular expression R :

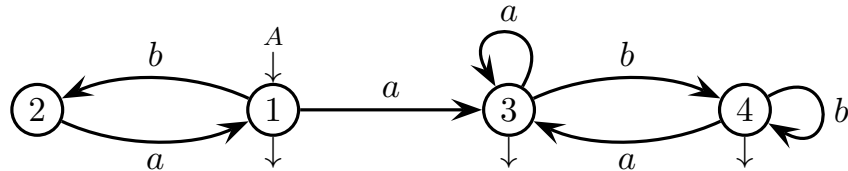
$$R_{\#} = ((a_1^+ b_2^*)^* \mid b_3 a_4)^+$$

initials	$a_1 b_3 \dashv$
terminal	followers
a_1	$a_1 b_2 b_3 \dashv$
b_2	$a_1 b_2 b_3 \dashv$
b_3	a_4
a_4	$a_1 b_3 \dashv$



Automaton A is clean and has four states. It may be non-minimal, anyway.

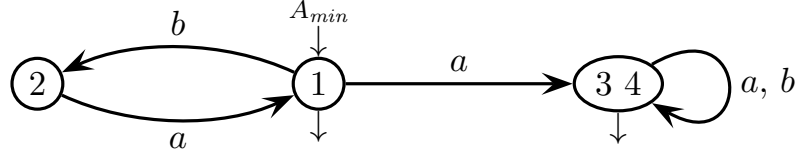
- (c) After state renaming, automaton A becomes as follows:



Since automaton A is deterministic, minimization theory is applicable:

- state 2 is non-final, hence it is distinguishable from the final states 1, 3 and 4
- state 1 goes to a non-final state by input b , while states 3 and 4 do not, hence state 1 is distinguishable from states 3 and 4
- both states 3 and 4 are final and make a closed group under a -transitions and b -transitions, hence they are undistinguishable

Therefore, automaton A is non-minimal. States 3 and 4 can be merged, thus resulting into the minimal automaton A_{min} below:



Incidentally, the minimal automaton makes it evident that the regular expression R is equivalent to the simpler regular expression below:

$$(b a)^* [a \Sigma^*]$$

Square brackets indicate optionality. This shorter form can be immediately obtained by eliminating node 2 and node (3 4).

- (d) Here is a right-unilinear grammar G equivalent to the regular expression R , obtained from the minimal automaton A_{min} (axiom 1):

$$G \left\{ \begin{array}{l} \langle 1 \rangle \rightarrow a \langle 34 \rangle \mid b \langle 2 \rangle \mid \varepsilon \\ \langle 2 \rangle \rightarrow a \langle 1 \rangle \\ \langle 34 \rangle \rightarrow a \langle 34 \rangle \mid b \langle 34 \rangle \mid \varepsilon \end{array} \right.$$

The nonterminal alphabet of grammar G is $V_N = \{ \langle 1 \rangle, \langle 2 \rangle, \langle 34 \rangle \}$.

2 Free Grammars and Pushdown Automata 20%

1. Given the two-letter terminal alphabet $\Sigma = \{a, b\}$, consider the language L over Σ defined below:

$$L = \{ w w^R \mid w \in L((a a \mid b)^*) \}$$

where string w^R is the reflection of the string w generated by this regular expression:

$$(a a \mid b)^*$$

Examples of strings of language L :

$$\varepsilon \quad a a a a \quad b b \quad a a b b a a$$

Counterexamples:

$$a a a \quad a b a a a b$$

Answer the following questions (all the grammars requested below must be BNF):

- (a) Write a non-ambiguous grammar G that generates language L .
- (b) Draw a syntax tree, according to grammar G , for the valid string:

$$a a b b a a$$

Argue that every string generated by grammar G has exactly one syntax tree.

- (c) Write a grammar G^* for the star closure L^* of language L .
 - (d) (optional) Say if the star closure grammar G^* of point (c) is ambiguous or not, and explain why.
-

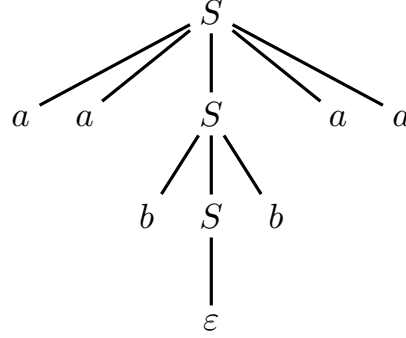
Solution

- (a) By observing that the regular language $L((aa \mid b)^*)$ is generated by the right-linear grammar $S \rightarrow aaS \mid bS \mid \varepsilon$, here is a possible grammar G that generates language L :

$$G \left\{ \begin{array}{l} S \rightarrow aaSa \\ S \rightarrow bSb \\ S \rightarrow \varepsilon \end{array} \right.$$

Grammar G is obtained by taking the palindromic versions of the right-linear rules above, i.e., for a right-linear rule like $X \rightarrow uY$ take the palindromic rule $X \rightarrow uY u^R$. Grammar G is not ambiguous (see the next point).

- (b) Here is the syntax tree of the valid string $aaabbaa$:



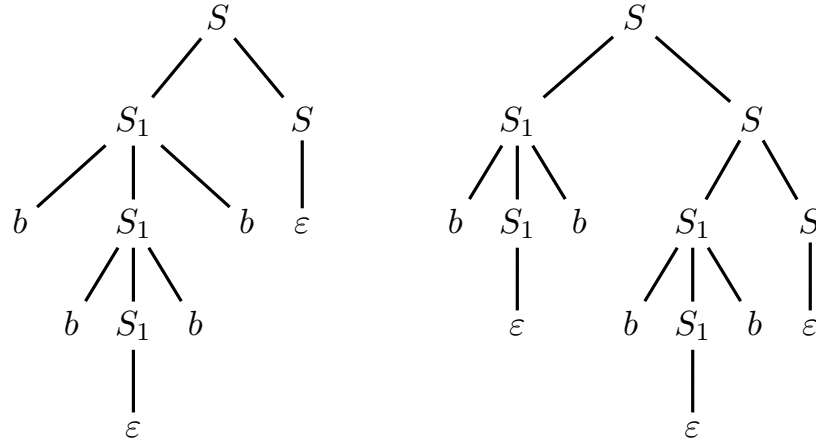
Grammar G is evidently non-ambiguous, since the substrings aa and b in the first half of any valid string orderly determine which of the two non-null rules of G must be used for the tree. Hence every valid string has exactly one tree.

- (c) The standard star-closure construction of a language yields a possible grammar G^* (axiom S), as follows:

$$G^* \left\{ \begin{array}{l} S \rightarrow S_1 S \\ S \rightarrow \varepsilon \\ S_1 \rightarrow aaS_1aa \\ S_1 \rightarrow bS_1b \\ S_1 \rightarrow \varepsilon \end{array} \right.$$

Grammar G^* instances as many nonterminals S_1 as the number of palindrome substrings is, and each nonterminal S_1 generates one palindrome substring.

- (d) The grammar G^* above is ambiguous. For instance, the valid string $b^4 \in L^*$ can be modeled both as a single palindrome, i.e., $(bb)(bb)^R$ (with $w = bb$), or as the concatenation of two identical palindromes, i.e., $(bb^R)(bb^R)$ (though trivially, with $w = b$). Consequently, according to G^* , such a string b^4 has two syntax trees, namely:



Longer strings of L^* may be modeled in more and more numerous ways, and thus have many more trees.

For teaching purposes, in addition to the exercise. Language L^* is likely to be inherently ambiguous, as it is a concatenation of independent palindromes, i.e., $x = w_1 w_1^R \cdot w_2 w_2^R$, etc. Yet, if for instance it happens that $w_2 = w_1$, then the string x can also be modeled as a single palindrome, i.e., $x = w_1 w_1^R w_1 w_1^R = (w_1 w_1^R) (w_1 w_1^R)^R$. However, in a purely syntactic way it is impossible to check if a substring (out of an infinite set of substrings) is repeated. Therefore no grammar of language L^* can check if a repetition occurs, and consequently both (or more) decompositions are always possible, with two (or more) different trees.

2. A programming language fragment includes expressions and variables that represent sets of atomic values identified by their names, and assignments of such expressions to the variables, according to the following rules:

- A set expression can be one of the following objects:
 - a variable name, i.e., an identifier, modeled by the terminal symbol `id`, not to be further expanded
 - a list, possibly empty, of comma-separated identifiers that denote the set elements, enclosed between the keywords `set` and `endset`; the identifiers of set elements are modeled by the terminal symbol `id`, not to be further expanded
 - the union (keyword `union`) or the difference (keyword `diff`) or the intersection (keyword `inters`) of set (sub)expressions; all these operators are binary; union and difference are *left*-associative, while intersection is *right*-associative; the precedence between these operators is, in increasing order: union, difference and intersection
 - a pair of round brackets “ () ” that enclose a set (sub)expression; by using round brackets one can change the default precedence defined between the set operators, see the fourth assignment in the example below
- An assignment statement, with the operator “`:=`”, assigns a set expression (on the right-hand side) to a variable (on the left-hand side).
- A sentence of the language is a non-empty list of assignment statements, and each statement is terminated by a semicolon.

Here is a sample sentence of the language with four assignments (the comments starting with `//` should be ignored):

```
s0 := set endset;           // empty set
s1 := set gray, white, black, red, green endset;    // non-empty set
s2 := s0 union set green endset;
s3 := s1 diff ( s2 union set blue endset );
```

Answer the following questions:

- (a) Write a grammar, of type *EBNF* and not ambiguous, that generates the above sketched language.
- (b) Draw the syntax subtree for the following assignment instruction:

```
s0 := set endset
```

- (c) Draw the syntax subtree for the following assignment instruction:

```
s2 := s0 union set green endset
```

- (d) (optional) Draw the syntax subtree for the following assignment instruction:

```
s4 := s1 inters ( s2 union s3 )
```

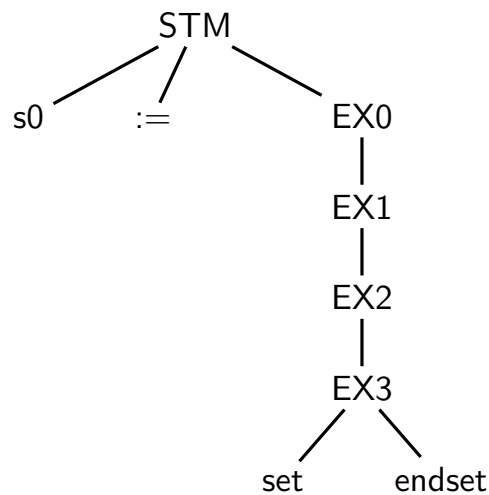

Solution

(a) Here is a viable grammar, of type EBNF and not ambiguous (axiom PRG):

$$\left\{ \begin{array}{l} \langle \text{PRG} \rangle \rightarrow (\langle \text{STM} \rangle \text{ “;”})^+ \\ \langle \text{STM} \rangle \rightarrow \text{id “:=” } \langle \text{EX0} \rangle \\ \langle \text{EX0} \rangle \rightarrow \langle \text{EX0} \rangle \text{ union } \langle \text{EX1} \rangle \mid \langle \text{EX1} \rangle \\ \langle \text{EX1} \rangle \rightarrow \langle \text{EX1} \rangle \text{ diff } \langle \text{EX2} \rangle \mid \langle \text{EX2} \rangle \\ \langle \text{EX2} \rangle \rightarrow \langle \text{EX3} \rangle \text{ inters } \langle \text{EX2} \rangle \mid \langle \text{EX3} \rangle \\ \langle \text{EX3} \rangle \rightarrow \text{id} \mid \text{set } [\langle \text{IDS} \rangle] \text{ endset} \mid \text{ “(” } \langle \text{EX0} \rangle \text{ “)”} \\ \langle \text{IDS} \rangle \rightarrow \text{id (“,” id })^* \end{array} \right.$$

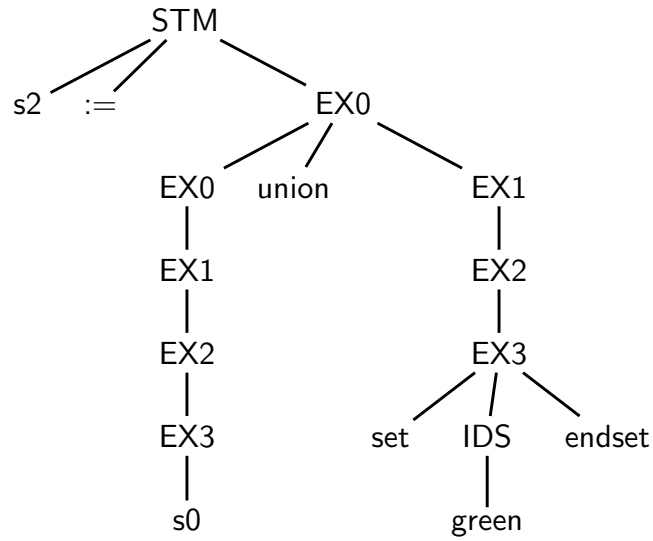
Recursion is on the left or right depending on associativity.

(b) Here is the syntax subtree of the statement “s0 := set endset”:



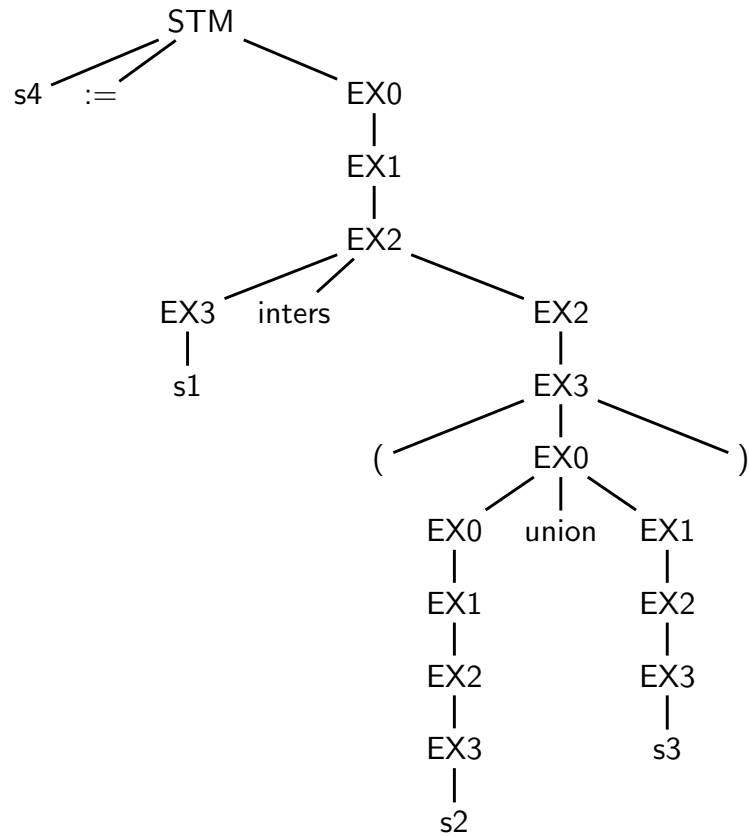
The objects schematized by terminal id are here replaced with their actual names.

(c) Here is the syntax subtree of the statement “s1 := s0 union set green endset”:



The objects schematized by terminal id are here replaced with their actual names.

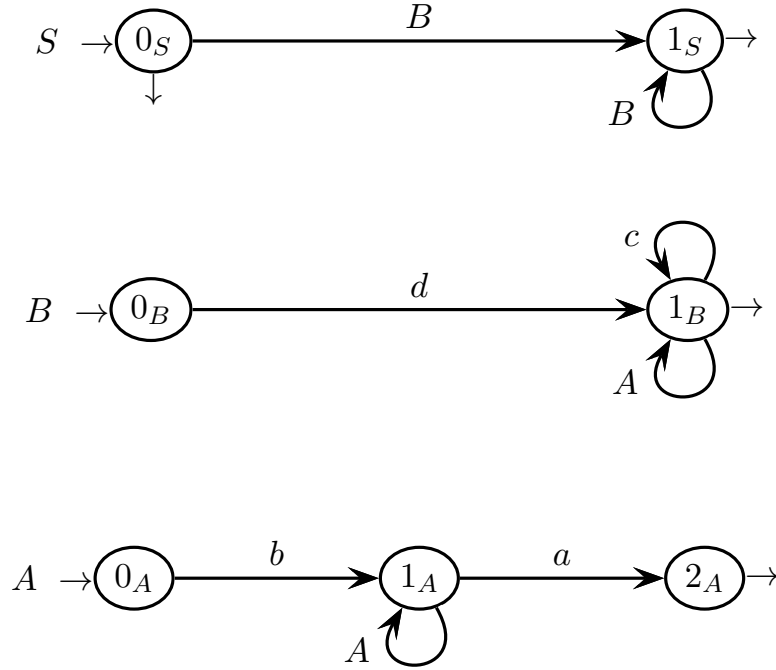
(d) Here is the syntax subtree of the statement “s4 := s1 inters (s2 union s3)”:



The objects schematized by terminal id are here replaced with their actual names.

3 Syntax Analysis and Parsing Methodologies 20%

1. Consider the grammar G defined by the following machine net, with the three-symbol nonterminal alphabet $V_N = \{ A, B, S \}$ (axiom S), and the four-letter terminal alphabet $\Sigma = \{ a, b, c, d \}$:



Answer the following questions (use the figures / tables / spaces on the next pages):

- (a) Do the two strings below:

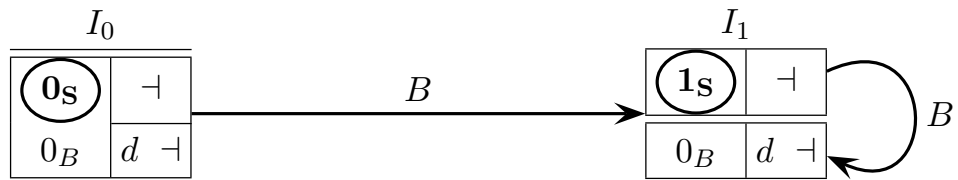
$d b a \quad d b b a c$

belong to language $L(G)$? If so, draw the corresponding syntax tree(s).

- (b) Complete the pilot automaton for grammar G (a few states are already given).
- (c) Say if grammar G is of type ELR(1), and justify your answer.
- (d) Write all the guide sets of the machine net of grammar G (on the call arcs and exit arrows). Say if grammar G is of type ELL(1), and justify your answer.
- (e) (optional) What is the smallest language family to which language $L(G)$ belongs? Justify your answer.

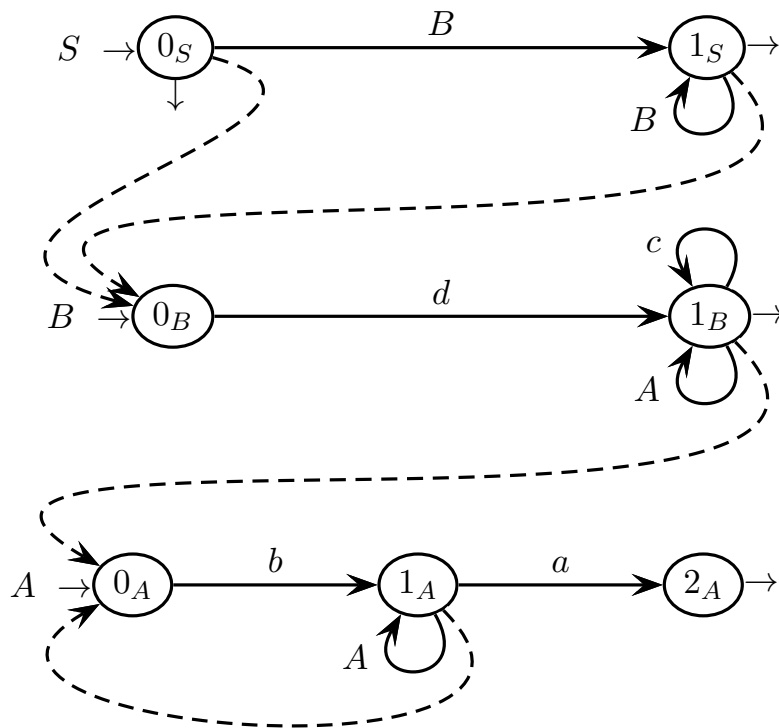
question (a) – syntax tree(s)

question (b) – draft pilot of grammar G to be completed



question (c) – space for answering

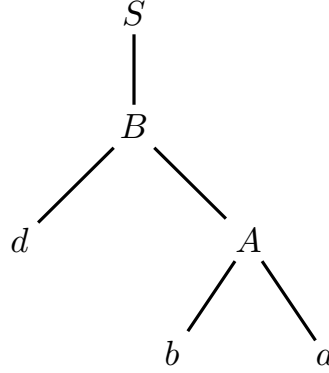
question (d) – guide sets (on all the call arcs and exit arrows) and answer



question (e) – space for answering

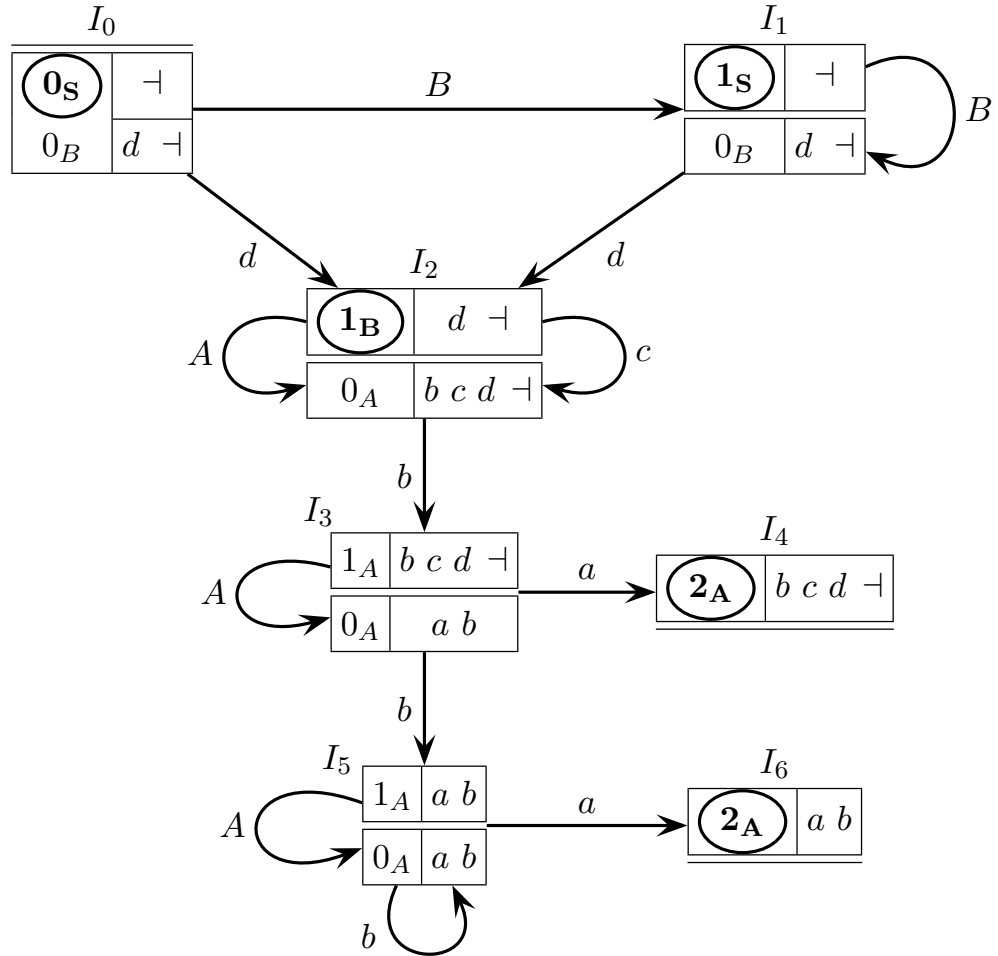
Solution

- (a) String dba belongs to language $L(G)$, and its syntax tree is as follows:



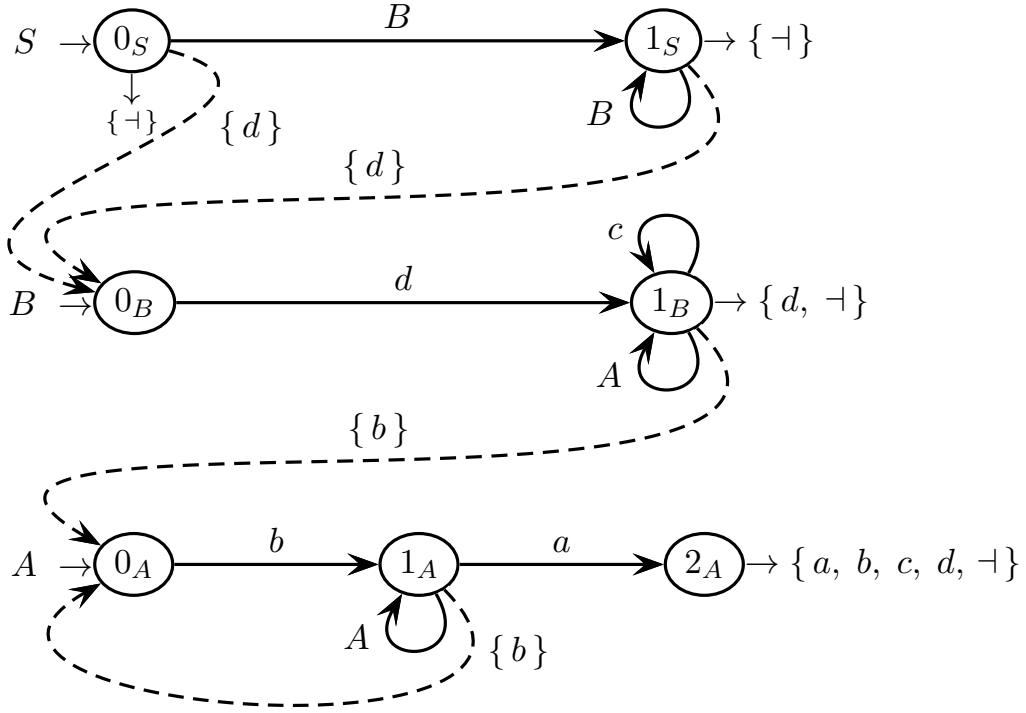
String $dbbac$ does not belong to language $L(G)$, as it contains unbalanced letters b and a (see also point (e)).

- (b) Here is the pilot automaton of grammar G :



The pilot has seven m-states in total. Notice that all the transitions between m-states are single (in fact the bases of the m-states are singletons).

- (c) There are not any SR or RR conflicts: the final states occur only in trap m-states (which do not have any outgoing shift arcs) or have the terminator as their look-ahead, and there are no m-states that contain two or more final states. Since the pilot has the STP, there is no convergence and consequently there are no convergence conflicts either. Therefore grammar G is of type ELR (1).
- (d) Here are the guide sets on the machine net of grammar G :



The fork points are on the following states: 0_S (double), 1_S (double), 1_B (triple) and 1_A (double), but in all such cases the guide sets are disjoint, thus there are no conflicts. Therefore grammar G is of type ELL (1).

- (e) Language $L(G)$ is infinite and context-free, but it is not regular. In fact, it holds:

$$L(A) = \{ b^n a^n \mid n \geq 1 \}$$

then:

$$L(B) = d (L(A) \mid c)^* = d (L(A))^* c^* (L(A))^* c^* \dots$$

and finally:

$$L(S) = (L(B))^+ = \underbrace{d (L(A))^* c^* (L(A))^* c^* \dots d (L(A))^* c^* \dots}_{L(B)} \dots$$

Thus a string of language $L(G) = L(S)$ is a sequence of arbitrarily many nested structures $b^{n_1} a^{n_1}, b^{n_2} a^{n_2}, \dots, b^{n_i} a^{n_i}, \dots$ (with $i \geq 1$ and $n_i \geq 1$), possibly separated by letters d and c . In any case, each nested structure $b^{n_i} a^{n_i}$ is non-regular. Therefore the smallest family of language $L(G)$ is the context-free one.

4 Language Translation and Semantic Analysis 20%

1. Consider a source language L^s of simple arithmetic expressions, with only addition “+”. Variables and numbers are schematized by the terminals v and n , respectively. Brackets are not allowed. Here is a grammar G^s for language L^s (axiom E):

$$G^s \left\{ \begin{array}{l} E \rightarrow v + E \\ E \rightarrow n + E \\ E \rightarrow v \\ E \rightarrow n \end{array} \right.$$

Answer the following questions:

- (a) Consider a syntactic translation τ_1 that introduces (round) brackets into the source expression of language L^s , so as to associate addition on the right. For instance:

$$\tau_1(v + v + n + v) = \left(v + \left(v + (n + v) \right) \right)$$

Write a translation grammar (or scheme) G_1^T for translation τ_1 . It is forbidden to change the source grammar G^s . Test the correctness of grammar G_1^T by drawing the source and destination trees of the sample source string:

$$v + v + n + v$$

- (b) Consider a syntactic translation τ_2 that introduces (round) brackets into the source expression of language L^s , so as to associate the addition of variables on the left, while the addition of numbers is not associated. For instance:

$$\tau_2(v + v + n + v) = \left((v + v) + n + v \right)$$

Write a translation grammar (or scheme) G_2^T for translation τ_2 . To this end, it is suggested to change the source grammar G^s . Test the correctness of grammar G_2^T by drawing the source and destination trees of the sample source string:

$$v + v + n + v$$

- (c) (optional) Is any of the translation grammars (or schemes) G_1^T and G_2^T deterministic of type LL or ELL, for some $k \geq 1$? For each grammar (or scheme) that is so, write the corresponding recursive-descent syntactic translator.

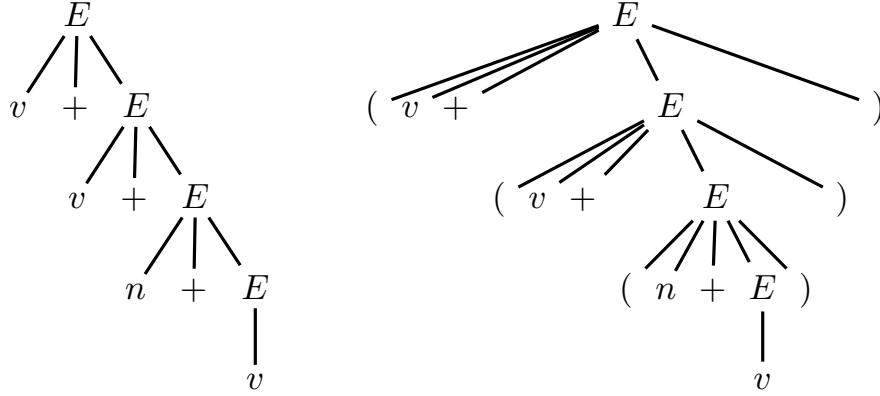
Solution

(a) Here is a possible translation grammar G_1^τ :

$$G_1^\tau \left\{ \begin{array}{l} E \rightarrow \left(\frac{\varepsilon}{v} \frac{\pm}{+} E \frac{\varepsilon}{+} \right) \quad // \text{ or also } E \rightarrow \frac{v \pm}{(v \pm)} E \frac{\varepsilon}{+} \\ E \rightarrow \left(\frac{\varepsilon}{n} \frac{\pm}{+} E \frac{\varepsilon}{+} \right) \quad // \text{ or also } E \rightarrow \frac{n \pm}{(n \pm)} E \frac{\varepsilon}{+} \\ E \rightarrow \frac{v}{v} \\ E \rightarrow \frac{n}{n} \end{array} \right.$$

As requested, the source grammar is still the original one G^s .

Here are the trees of the source string $v + v + n + v$ (source and destination):

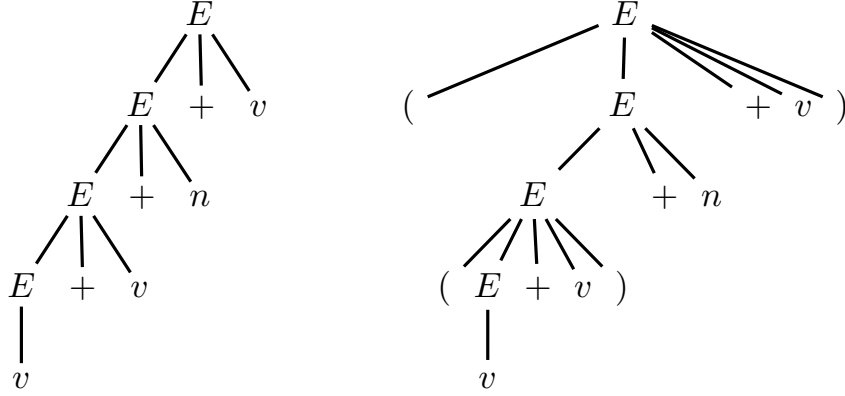


(b) Of course, recursion must be turned onto the left for the rule $E \rightarrow v + E$ of the variables, i.e., $E \rightarrow E + v$, and then the rule must be bracketed. As for the numbers, their rule $E \rightarrow n + E$ must also be made left-recursive, i.e., $E \rightarrow E + n$, for otherwise the source language would be changed (if this rule were left right-recursive, all the variables would always occur on the right of all the numbers), but no brackets are generated. Here is a possible translation grammar G_2^τ :

$$G_2^\tau \left\{ \begin{array}{l} E \rightarrow \left(\frac{\varepsilon}{+} E \frac{\pm}{+} \frac{v}{v} \frac{\varepsilon}{+} \right) \quad // \text{ or also } E \rightarrow \frac{\varepsilon}{+} E \frac{\pm v}{+v} \\ E \rightarrow E \frac{\pm}{+} \frac{n}{n} \quad // \text{ or also } E \rightarrow E \frac{\pm n}{+n} \\ E \rightarrow \frac{v}{v} \\ E \rightarrow \frac{n}{n} \end{array} \right.$$

The source grammar of G_2^τ is different from the original source grammar G^s : the two top rules have become right-recursive. Notice however that the nonterminal structures of grammars G_1^τ and G_2^τ are the same: by canceling all the terminals the two grammars become identical (though they become trivial).

Here are the trees of the source string $v + v + n + v$ (source and destination):



- (c) The source grammar G^s (still the original one) of G_1^T is of type $LL(2)$, therefore translation τ_1 is deterministic of type $LL(2)$. In fact, with $k = 2$ the guide sets of the four rules of G^s are the singletons $\{v+\}$, $\{n+\}$, $\{v-\}$ and $\{n-\}$, which are all different from one another. The source grammar of G_2^T (which is different from the original G^s , as said) is left recursive, thus it is neither LL nor ELL for any $k \geq 1$, and is not deterministic in the requested way (it may still be deterministic of type ELR , but this is not requested here).

Here is the recursive-descent translator for G_1^T , with all the errors grouped at the end, and a procedure *shift* that shifts of one position the text window $\langle cc, lc \rangle$ made of the current and look-ahead characters:

```

char cc, lc                                // current and look-ahead characters
procedure shift                             // header only - shift cc and lc
procedure E
begin
  switch cc lc do                           // select guide
  case v + do
    write ( v
    shift                                     // shift cc and lc
    write +
    shift                                     // shift cc and lc
    if cc ∈ { v, n } then
      call E
      write )
      if cc = ⊥ then return
    break
  case n + do
    write ( n
    shift                                     // shift cc and lc
    write +
    shift                                     // shift cc and lc
    if cc ∈ { v, n } then
      call E
      write )
      if cc = ⊥ then return
    break
  case v ⊥ do
    write v
    cc := lc                                // input end - shift only cc
    if cc = ⊥ then return
    break
  case n ⊥ do
    write n
    cc := lc                                // input end - shift only cc
    if cc = ⊥ then return
  error                                     // any syntactic error

```

This is a very formal and complete encoding, where the look-ahead window is shifted one character at a time, and all the checks are performed. A few checks are actually redundant and can be removed, thus shortening the code. Furthermore, the text window $\langle cc, lc \rangle$ made of the current and look-ahead characters can be shifted of two positions at once, by a procedure *shift2*:

```

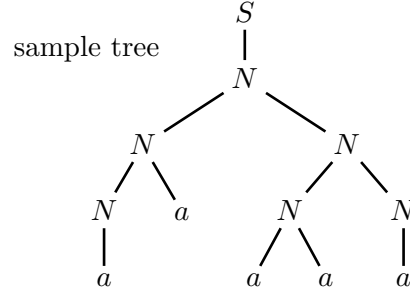
char cc, lc                                // current and look-ahead characters
procedure shift2                            // header only - double shift cc and lc
procedure E
begin
  switch cc lc do                             // select guide
  case v + do
    write ( v +
    shift2                                     // double shift cc and lc
    if cc ∈ { v, n } then
      call E
      write )
      if cc = ⊥ then return
    break
  case n + do
    write ( n +
    shift2                                     // double shift cc and lc
    if cc ∈ { v, n } then
      call E
      write )
      if cc = ⊥ then return
    break
  case v ⊥ do
    write v
    return
  case n ⊥ do
    write n
    return
  error                                     // any syntactic error

```

Of course other encodings are possible, more or less optimized.

2. Consider the following abstract syntax (nonterminals S, N – terminals a – axiom S):

$$\left\{ \begin{array}{l} 1: S \rightarrow N \\ 2: N \rightarrow N N \\ 3: N \rightarrow N a \\ 4: N \rightarrow a a \\ 5: N \rightarrow a \end{array} \right.$$

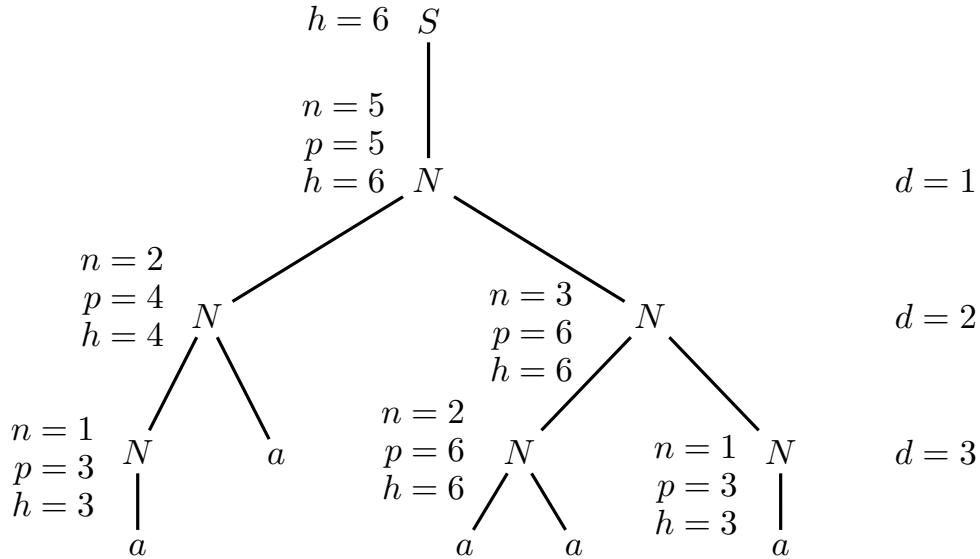


The grammar is clearly unsuitable (ambiguous) for syntax analysis, but in what follows it is assumed the existence of a syntax tree for every string of the language.

For the tree root, one wants to compute the highest value assumed, over all the tree nodes, by the product of the depth of the node and the number of leaves of the subtree rooted at that node. To this end, these four attributes n, d, p and h are introduced:

- for each node of type N :
 - a *left* attribute n equal to the number of leaves a in the subtree rooted at N
 - a *right* attribute d equal to the distance from the root S to N (note that $d \geq 1$)
 - a *left* attribute p equal to the product of the two attributes d and n of N
- for each node of type N and the root S : a *left* attribute h equal to the highest value that attribute p takes over all the nodes of the (sub)tree rooted at N or S

Here is the sample tree decorated with attributes (attribute d is factored on the right):



Answer the following questions (use the tables / trees / spaces on the next pages):

- Define the semantic rules for computing attribute n .
- Define the semantic rules for computing attributes d and p .
- Define the semantic rules for computing attribute h .
- (optional) Argue that the so-defined attribute grammar is of type one-sweep.

table to be filled in for question (a)

syntax

semantic functions

1: $S_0 \rightarrow N_1$

2: $N_0 \rightarrow N_1 N_2$

3: $N_0 \rightarrow N_1 a$

4: $N_0 \rightarrow a a$

5: $N_0 \rightarrow a$

table to be filled in for question (b) – do not repeat the previous functions

syntax

semantic functions

1: $S_0 \rightarrow N_1$

2: $N_0 \rightarrow N_1 N_2$

3: $N_0 \rightarrow N_1 a$

4: $N_0 \rightarrow a a$

5: $N_0 \rightarrow a$

table to be filled in for question (c) – do not repeat the previous functions

#	<i>syntax</i>	<i>semantic functions</i>
1:	$S_0 \rightarrow N_1$	
2:	$N_0 \rightarrow N_1 N_2$	
3:	$N_0 \rightarrow N_1 a$	
4:	$N_0 \rightarrow a a$	
5:	$N_0 \rightarrow a$	

question (d) – space for answering

Solution

(a) Semantic functions of attribute n :

#	<i>syntax</i>	<i>semantic functions</i>
1:	$S_0 \rightarrow N_1$	
2:	$N_0 \rightarrow N_1 N_2$	$n_0 := n_1 + n_2$
3:	$N_0 \rightarrow N_1 a$	$n_0 := n_1 + 1$
4:	$N_0 \rightarrow a a$	$n_0 := 2$
5:	$N_0 \rightarrow a$	$n_0 := 1$

(b) Semantic functions of attributes d and p :

#	<i>syntax</i>	<i>semantic functions</i>
1:	$S_0 \rightarrow N_1$	$d_1 := 1$
2:	$N_0 \rightarrow N_1 N_2$	$d_1 := d_0 + 1 \quad d_2 := d_0 + 1$ $p_0 := n_0 \times d_0$
3:	$N_0 \rightarrow N_1 a$	$d_1 := d_0 + 1$ $p_0 := n_0 \times d_0$
4:	$N_0 \rightarrow a a$	$p_0 := n_0 \times d_0$
5:	$N_0 \rightarrow a$	$p_0 := n_0 \times d_0$

(c) Semantic functions of attribute h :

#	<i>syntax</i>	<i>semantic functions</i>
1:	$S_0 \rightarrow N_1$	$h_0 := h_1$
2:	$N_0 \rightarrow N_1 N_2$	$h_0 := \max(p_0, h_1, h_2)$
3:	$N_0 \rightarrow N_1 a$	$h_0 := \max(p_0, h_1)$
4:	$N_0 \rightarrow a a$	$h_0 := p_0$
5:	$N_0 \rightarrow a$	$h_0 := p_0$

(d) The attribute grammar is of type one-sweep. In fact, for each rule it holds:

- there are no cycles in the dependency graph
- no right attribute depends on a left attribute

One can further observe that no attribute of a node depends on attributes of any siblings of that node, hence the sibling (brother) graph is empty.